ESCENARIOS DE PRUEBA QA – GRUPO 2

API y Seguridad | Checklist de seguimiento por prioridad

#	Prioridad	ID	Área	Escenario de prueba	Resultado esperado	Seguimiento / Notas	Estado	Evidencias
1	**ALTA**	G2-01	Login	Iniciar sesión con usuario y contraseña correctos	Permite acceso y crea sesión/token válido	Se realizaron pruebas de inicio de sesión utilizando las seis cuentas exclusivas de QA proporcionadas por el PM: administrador, coordinador, docente, revisor, autoridad y auditor. Todas las solicitudes al endpoint POST /api/v1/auth/login devolvieron HTTP 200, access_token, token_type bearer y expires_in de 7200 segundos. Los datos de usuario y roles coincidieron con las cuentas utilizadas y no se observaron contraseñas ni hashes en las respuestas. Los seis inicios de sesión fueron exitosos. Queda pendiente verificar el contenido y funcionamiento de los JWT en endpoints protegidos. Como prueba complementaria solicitada por el PM, se verificó el inicio de sesión de una cuenta QA inactiva. La API respondió HTTP 401 (Unauthorized), sin generar un token de acceso y utilizando el mensaje genérico «Correo o contraseña incorrectos». El comportamiento fue el esperado.	EN PROCESO	Evidencias
2	**ALTA**	G2-02	Login	Iniciar sesión con contraseña incorrecta	Rechaza acceso con 401	Se realizó una prueba de inicio de sesión utilizando la cuenta QA de administrador con una contraseña incorrecta. La solicitud POST /api/v1/auth/login devolvió HTTP 401 (Unauthorized), con el código UNAUTHORIZED y el mensaje genérico «Correo o contraseña incorrectos». La respuesta no incluyó access_token ni permitió iniciar sesión. El resultado coincide con el comportamiento esperado, por lo que el escenario se considera aprobado.	PASS	Evidencias
3	**ALTA**	G2-05	Sesión	Usar sesión/token después de cerrar sesión	El acceso debe ser rechazado	Se revisó la documentación Swagger y el código fuente de la API para verificar el mecanismo de cierre de sesión. En Swagger únicamente aparece POST /api/v1/auth/login dentro de Autenticación. Posteriormente, se revisó g2_api/routers/auth.py y se confirmó que el login registra el hash del JWT en la tabla sesion_token, con una referencia a la futura revocación mediante revocado_en. Sin embargo, no se encontró una ruta de logout implementada en el router de autenticación. Por esta razón, no fue posible cerrar una sesión ni comprobar que el JWT anterior sea rechazado después del logout. El escenario queda BLOQUEADO hasta que se implemente o se indique un mecanismo funcional de cierre de sesión.	BLOQUEADO / NO IMPLEMENTADO EN LA VERSIÓN REVISADA	Evidencias
4	**ALTA**	G2-06	Autenticación	Acceder a un endpoint protegido sin iniciar sesión	Rechaza acceso con 401	Se realizaron pruebas de acceso a GET /api/v1/users sin autenticación y mediante diferentes encabezados. Sin enviar credenciales, la API devolvió HTTP 422 (VALIDATION_ERROR) por ausencia del encabezado usuario-actual, en lugar del HTTP 401 esperado. Al utilizar un identificador inexistente o el correo del administrador QA sin JWT, respondió HTTP 401, rechazando el acceso. Posteriormente, se envió un JWT del administrador mediante Authorization: Bearer, confirmando que tenía tres segmentos y ningún carácter inválido; sin embargo, la API respondió HTTP 422. La revisión del código confirmó que el middleware identifica usuarios mediante el encabezado usuario-actual y una lista temporal cargada desde JSON, mientras que el login utiliza la base de datos y genera JWT. Se confirma una incompatibilidad de integración entre el login y el endpoint protegido. No se obtuvo acceso a información protegida.	FAIL	Evidencias

5	**ALTA**	G2-08	Permisos	Usuario autenticado intenta ejecutar una acción para la que no tiene permiso	Rechaza con 403	Se intentó verificar el control de permisos mediante una solicitud GET /api/v1/users utilizando el JWT de la cuenta QA docente, cuyo rol es teacher. El resultado esperado era HTTP 403 (Forbidden), al tratarse de un endpoint reservado para administradores. Sin embargo, la API devolvió HTTP 422 (Unprocessable Content). La revisión previa del código confirmó que el middleware requiere el encabezado usuario-actual y utiliza una lista temporal de usuarios en lugar de autenticar mediante JWT. Por esta incompatibilidad, no fue posible comprobar si el sistema aplica correctamente las restricciones de permisos. El escenario queda BLOQUEADO hasta que se integre la autenticación Bearer con los endpoints protegidos.	BLOQUEADO — Fallo de integración JWT	Evidencias
6	**ALTA**	G2-09	Roles	Docente intenta acceder a funciones exclusivas de administrador	Acceso rechazado	Se revisaron los endpoints disponibles en Swagger para identificar funciones administrativas y comprobar las restricciones de acceso del rol docente. Se utilizó como referencia la prueba realizada con el JWT de la cuenta QA docente sobre GET /api/v1/users, endpoint restringido al rol administrador. La API respondió HTTP 422 en lugar del HTTP 403 esperado. La revisión del middleware confirmó que el endpoint requiere el encabezado usuario-actual y utiliza una lista temporal de usuarios, sin integrar la autenticación mediante JWT. Por esta razón, no fue posible validar correctamente la restricción de permisos para el rol docente. El escenario queda BLOQUEADO hasta que se complete la integración del middleware.	BLOQUEADO — Integración JWT pendiente	Evidencias
7	**ALTA**	G2-10	Roles / Backend	Saltarse el frontend e intentar ejecutar directamente una acción prohibida	El backend rechaza la operación	Se realizó una solicitud directa al endpoint GET /api/v1/roles utilizando únicamente el encabezado usuario-actual con el correo de un administrador activo del archivo usuarios_prueba.json, sin contraseña ni token JWT. El servidor respondió HTTP 200, permitiendo el acceso al recurso sin validar la autenticación mediante Bearer JWT. Se evidencia una debilidad en el control de autenticación del backend, ya que confía en una identidad proporcionada por el cliente.	FAIL	Evidencias
8	**ALTA**	G2-15	Seguridad XSS	Introducir contenido con código/script en campos de texto	El código no se ejecuta en ninguna pantalla	Se realizó una prueba de Cross-Site Scripting (XSS) en el formulario de inicio de sesión del Grupo 2, utilizando la cadena `<script>alert('QA-XSS')</script>` en el campo Usuario. El frontend, ejecutado desde http://localhost:5500, envió la solicitud a la API y recibió HTTP 401 (Unauthorized). El navegador mostró únicamente el mensaje «Correo o contraseña incorrectos», sin ejecutar el script introducido. Adicionalmente, se revisó el código del endpoint de autenticación y se confirmó que utiliza mensajes de error genéricos. El caso evaluado cumple con el comportamiento esperado; no se han evaluado otros formularios o pantallas del sistema.	PASS (caso evaluado)	Evidencias
9	**ALTA**	G2-16	Seguridad SQL	Probar entradas orientadas a SQL Injection	La entrada no altera consultas ni la base de datos	Se realizó una prueba de inyección SQL en el endpoint POST `/api/v1/auth/login`, utilizando la cadena `' OR '1'='1` en el campo correo. La API respondió HTTP 401 (Unauthorized), rechazando la autenticación. Adicionalmente, se revisó el código del endpoint y se confirmó que utiliza consultas mediante SQLAlchemy, sin concatenar directamente la entrada del usuario en sentencias SQL. No se identificó una vulnerabilidad de inyección SQL en el caso evaluado. La verificación no abarca todos los endpoints ni confirma por sí sola la ausencia de modificaciones en la base de datos.	PASS (caso evaluado)	Evidencias

10	**ALTA**	G2-17	Autorización	Cambiar un identificador para intentar acceder a recursos de otro usuario	El servidor verifica autorización y bloquea el acceso	Se revisó el endpoint PATCH /api/v1/users/{usuario_id} y se ejecutó una solicitud hacia el ID inexistente 999999, enviando un cuerpo JSON vacío y únicamente el encabezado usuario-actual con un administrador del archivo usuarios_prueba.json, sin token JWT. El backend respondió HTTP 404 (Usuario no encontrado), demostrando que la solicitud superó el control de acceso y alcanzó la búsqueda del recurso sin autenticación JWT. No se modificaron registros. Se confirma una deficiencia de autenticación y autorización en la ruta administrativa, relacionada con G2-10. No fue posible confirmar un IDOR entre usuarios porque solo existe un usuario en los datos temporales y no se realizó una operación sobre recursos ajenos.	PARCIAL — Hallazgo de seguridad confirmado	Evidencias
11	**ALTA**	G2-19	Archivos	Intentar subir un tipo de archivo no permitido	Archivo rechazado	Se verificó la disponibilidad de endpoints para carga de archivos mediante Swagger y la revisión del código fuente. Se confirmó que el módulo g2_api/routers/files.py no está implementado y que main.py únicamente incorpora dicho router cuando existe. Por lo tanto, no fue posible ejecutar una solicitud con un archivo de extensión no permitida ni comprobar si el backend aplica las restricciones correspondientes. El escenario queda bloqueado por falta de implementación del módulo de archivos.	PENDIENTE-FALTA IMPLEMENTAR	
12	**ALTA**	G2-20	Archivos	Intentar subir un archivo que exceda el tamaño máximo	Archivo rechazado	Se revisaron los endpoints disponibles en Swagger y el código fuente del Grupo 2, confirmando que no existe el módulo `g2_api/routers/files.py` ni un endpoint habilitado para la carga de archivos. En `main.py` se verificó que el router de archivos únicamente se incorpora cuando dicho módulo está disponible. Por lo tanto, no fue posible realizar una prueba con un archivo que excediera el tamaño máximo permitido ni validar la respuesta del backend ante esta situación. El escenario queda bloqueado por falta de implementación de la funcionalidad.	PENDIENTE-FALTA IMPLEMENTAR	
13	**ALTA**	G2-21	Archivos / Permisos	Intentar descargar o acceder al archivo de otro usuario sin autorización	Acceso rechazado	Se revisaron los endpoints disponibles en Swagger y el código fuente del Grupo 2, confirmando que no existe el módulo `g2_api/routers/files.py` ni un endpoint habilitado para la descarga de archivos. En `main.py` se verificó que el router de archivos únicamente se incorpora cuando dicho módulo está disponible. Por lo tanto, no fue posible intentar descargar un archivo perteneciente a otro usuario ni comprobar que el backend aplique las restricciones de autorización correspondientes. El escenario queda bloqueado por falta de implementación de la funcionalidad.	PENDIENTE-FALTA IMPLEMENTAR	
14	**ALTA**	G2-22	Archivos / Seguridad	Intentar manipular el nombre o ruta del archivo	No permite acceder ni escribir fuera del almacenamiento autorizado	Se revisaron los endpoints disponibles en Swagger y el código fuente del Grupo 2, confirmando que no existe el módulo `g2_api/routers/files.py` ni endpoints habilitados para subir, descargar o acceder a archivos. En `main.py` se verificó que el router de archivos únicamente se incorpora cuando dicho módulo está disponible. Por lo tanto, no fue posible realizar pruebas de manipulación de nombres o rutas de archivos (Path Traversal) ni comprobar si el backend bloquea el acceso a directorios no autorizados. El escenario queda bloqueado por falta de implementación de la funcionalidad.	PENDIENTE-FALTA IMPLEMENTAR	

15	**ALTA**	G2-23	Errores / Seguridad	Provocar errores para buscar información interna	No muestra SQL, rutas internas, stack traces ni secretos	Se realizaron dos solicitudes POST a `/api/v1/auth/login` para verificar que los mensajes de error no expongan información sensible. La primera utilizó un JSON mal formado y obtuvo HTTP 422 (VALIDATION_ERROR), con el detalle JSON_INVALID. La segunda envió valores numéricos en los campos correo y password, obteniendo HTTP 422 con errores STRING_TYPE para ambos campos. En las respuestas evaluadas no se encontraron contraseñas, tokens, consultas SQL, rutas internas ni trazas de ejecución. El sistema devolvió mensajes de error controlados y estructurados.	PASS (casos evaluados)	Evidencia
16	**ALTA**	G2-24	Información sensible	Revisar respuestas de API buscando información sensible	No expone contraseñas, secretos ni información sensible innecesaria	Se realizó una solicitud GET /api/v1/users y se verificó mediante un script de Python que la respuesta no incluyera campos sensibles conocidos, como password, hash_contrasena, password_hash, access_token o refresh_token. El backend respondió HTTP 200 y devolvió una lista con un usuario. La comprobación no identificó ninguno de los campos sensibles evaluados. Adicionalmente, se revisó el código de users.py, donde se eliminan los campos password y hash_contrasena antes de devolver la respuesta. El escenario cumple con el comportamiento esperado para el endpoint evaluado. Sin embargo, se mantiene la vulnerabilidad de acceso sin JWT documentada en G2-10.	PASS	Evidencias
17	**ALTA**	G2-35	Configuración	Revisar repositorio y configuración	No contiene contraseñas ni variables sensibles	Se verificó la protección de variables sensibles del Grupo 2. El archivo .gitignore contiene reglas para excluir .env y archivos relacionados. En la carpeta del proyecto revisada en GitHub únicamente se encontró . env.example, cuyos valores DATABASE_URL y SECRET_KEY utilizan indicaciones de configuración en lugar de credenciales reales. Adicionalmente, se realizó una búsqueda de patrones de posibles secretos en los archivos Python de g2_api, sin encontrar coincidencias. No se identificó exposición de credenciales en las comprobaciones realizadas.	PASS (comprobaciones realizadas)	Evidencias
18	**ALTA**	G2-37	Autenticación / Seguridad	Usar un token vencido para acceder a un endpoint protegido	Devuelve 401	Se revisó la validación de tokens JWT vencidos. En seguridad.py se identificó la función verificar_token(), que utiliza jwt.decode() con la clave y el algoritmo configurados, y captura jwt.InvalidTokenError para rechazar tokens inválidos o expirados. Mediante una búsqueda global se comprobó que verificar_token() aparece en seguridad.py, test_auth.py y GUIA_GRUPO2.md, pero no se encontraron llamadas desde auth_middleware.py ni desde los endpoints protegidos de usuarios y roles. Estos endpoints continúan utilizando el encabezado usuario-actual en lugar de validar el token Bearer. Por lo tanto, aunque existe la función auxiliar de validación, no fue posible comprobar que un JWT vencido produzca HTTP 401 al acceder a un recurso protegido. El escenario queda bloqueado por falta de integración de la autenticación JWT.	BLOQUEADO	Evidencias

19	**ALTA**	G2-38	Autenticación / Seguridad	Usar un token modificado manualmente para acceder a un endpoint protegido	Devuelve 401	Se verificó el rechazo de tokens JWT alterados mediante la función verificar_token() de seguridad.py. En test_auth.py se identificó la prueba test_token_alterado_no_es_valido, que obtiene un JWT válido, modifica su contenido agregando un carácter y comprueba que la función devuelva None. La prueba automatizada se ejecutó mediante pytest y finalizó correctamente (1 passed, 6 deselected). Sin embargo, los endpoints protegidos revisados continúan utilizando el encabezado usuario-actual y no integran la validación JWT. Por lo tanto, se confirma el funcionamiento de la validación auxiliar, pero no fue posible comprobar que una solicitud con un token alterado reciba HTTP 401. El escenario queda bloqueado por falta de integración del middleware JWT.	BLOQUEADO — Validación JWT no integrada en endpoints protegidos	Evidencias
20	**MEDIA**	G2-03	Login	Iniciar sesión con usuario inexistente	Acceso rechazado sin revelar información sensible	Se realizó una prueba de inicio de sesión utilizando un correo inexistente. Inicialmente, mediante una solicitud directa a la API, se obtuvo HTTP 401 (Unauthorized), con el mensaje genérico «Correo o contraseña incorrectos». Posteriormente, se repitió la prueba desde el frontend en http://localhost:5500, confirmando que el sistema muestra el mismo mensaje y que la consola registra HTTP 401. En ningún caso se reveló información sobre la existencia del usuario ni se permitió el acceso. El escenario cumple con el resultado esperado.	PASS	Evidencias
21	**MEDIA**	G2-04	Login / Validación	Enviar usuario y contraseña vacíos	Solicitud rechazada mediante validación	Se realizaron pruebas de inicio de sesión con campos vacíos tanto en la API como en el frontend. En la API, al enviar correo y contraseña vacíos, se obtuvo HTTP 401 (Unauthorized), rechazando el acceso. Posteriormente, desde http://localhost:5500, se comprobó que el formulario identifica los campos obligatorios: al dejar Usuario vacío, solicita completarlo; al ingresar un correo y dejar Contraseña vacía, muestra la misma validación. En ambos casos, el navegador impide enviar el formulario. Se cumple el resultado esperado de rechazar el inicio de sesión con campos vacíos. Como observación, la API devuelve 401 en lugar de un error de validación 422.	PASS	Evidencias
22	**MEDIA**	G2-07	Identidad	Consultar /me con un usuario autenticado	Devuelve usuario, roles y permisos correspondientes	Primero se realizó una búsqueda en el código fuente, sin encontrar una declaración de ruta para /me. Las referencias identificadas en CONTRATO.md, GUIA_GRUPO2.md, main.py y seguridad.py corresponden a documentación y comentarios, no a una implementación funcional. Posteriormente, se ejecutaron solicitudes GET a /api/v1/auth/me y /api/v1/me, obteniendo HTTP 404 (Not Found) en ambos casos. Adicionalmente, se comprobó que Swagger UI no presenta el endpoint requerido. Por lo tanto, no fue posible verificar la recuperación de la identidad y los roles del usuario autenticado. El escenario queda BLOQUEADO debido a que el endpoint no está disponible en la implementación evaluada.	BLOQUEADO — Endpoint no disponible.	Evidencia
23	**MEDIA**	G2-11	Usuarios	Consultar un usuario existente	Devuelve la información permitida correctamente	Se realizó una solicitud GET /api/v1/users utilizando el encabezado usuario-actual con un administrador activo del archivo usuarios_prueba.json. El servidor respondió HTTP 200 y devolvió una lista con un usuario existente. Sin embargo, la consulta fue permitida sin presentar un token JWT, evidenciando una deficiencia en el control de autenticación. Adicionalmente, no se encuentra disponible un endpoint GET /api/v1/users/{usuario_id} para consultar usuarios individualmente.	PARCIAL	Evidencias

24	**MEDIA**	G2-12	Usuarios	Consultar un ID de usuario inexistente	Responde 404	Se verificaron los endpoints disponibles en Swagger y el archivo users.py, confirmando que no existe GET /api/v1/users/{usuario_id}. Se realizó una solicitud GET /api/v1/users/999999 utilizando un ID inexistente y el backend respondió HTTP 405 (Method Not Allowed), debido a que la ruta parametrizada únicamente admite PATCH. No fue posible validar el comportamiento esperado de HTTP 404 para un usuario inexistente. El escenario queda bloqueado por falta del endpoint GET individual.	BLOQUEADO	Evidencias
25	**MEDIA**	G2-13	Validación	Enviar datos obligatorios vacíos	Responde 422 indicando los campos inválidos	Se ejecutó una solicitud POST /api/v1/auth/login enviando únicamente el campo correo y omitiendo el campo obligatorio password. El backend respondió HTTP 422 (Unprocessable Content), con código VALIDATION_ERROR y detalle password: MISSING (Field required). Se confirmó que la API valida correctamente la ausencia del campo obligatorio y devuelve un mensaje de error estructurado.	PASS	Evidencias
26	**MEDIA**	G2-14	Validación	Enviar datos con formato incorrecto	No procesa ni almacena información inválida	Se realizó una solicitud POST /api/v1/auth/login enviando el campo correo con un valor numérico (12345) en lugar de una cadena de texto. El backend respondió HTTP 422 (Unprocessable Content), con código VALIDATION_ERROR y detalle STRING_TYPE (Input should be a valid string). Se confirmó que la API rechaza correctamente tipos de datos incompatibles y devuelve un error de validación estructurado.	PASS	Evidencias
27	**MEDIA**	G2-18	Archivos	Subir un tipo de archivo permitido	Archivo aceptado y devuelve file_id	Se revisaron los endpoints disponibles en Swagger, el archivo main.py y la carpeta g2_api/routers. Se confirmó que no existe el módulo files.py ni se encuentra disponible un endpoint para cargar archivos. En main.py, líneas 39–45, se observa que el router de archivos únicamente se incorpora cuando dicho módulo existe. Por lo tanto, no fue posible ejecutar la prueba de subida de un archivo permitido ni verificar la respuesta esperada. El escenario queda bloqueado por falta de implementación.	PENDIENTE-FALTA IMPLEMENTAR	
28	**MEDIA**	G2-25	API / HTTP	Realizar correctamente una solicitud de creación	Utiliza el código HTTP correspondiente, por ejemplo 201	Se realizó una solicitud POST /api/v1/users para crear un usuario ficticio de prueba con datos válidos. El backend respondió HTTP 201 (Created), asignó automáticamente el ID 2 y devolvió los datos del usuario creado, incluyendo su rol y estado activo, sin exponer la contraseña ni su hash. Se confirmó que la funcionalidad de creación opera correctamente en la lista temporal de usuarios. Sin embargo, la operación fue autorizada utilizando únicamente el encabezado usuario-actual, sin presentar un token JWT, lo que confirma que la vulnerabilidad de autenticación identificada en G2-10 también afecta a operaciones administrativas de creación de usuarios.	PASS — funcionalidad de creación, con hallazgo de seguridad asociado.	Evidencias
29	**MEDIA**	G2-26	API / HTTP	Solicitar un recurso inexistente	Devuelve 404	Se verificó el comportamiento de la API al solicitar un recurso inexistente utilizando el ID 999999. La solicitud GET /api/v1/users/999999 devolvió HTTP 405 (Method Not Allowed), debido a que no existe un endpoint GET para consultar usuarios individualmente. Adicionalmente, se realizó una solicitud PATCH /api/v1/users/999999 con un cuerpo JSON vacío, obteniendo HTTP 404 (Not Found) y el mensaje "Usuario no encontrado". Se confirmó que el backend identifica correctamente un recurso inexistente mediante PATCH; sin embargo, no fue posible validar el comportamiento esperado mediante una consulta GET. El escenario queda parcialmente evaluado.	PARCIAL	

30	**MEDIA**	G2-27	API / Estados	Intentar crear un usuario con un correo ya registrado	Devuelve 409 y el usuario duplicado no se crea	Se verificó previamente mediante GET /api/v1/users que el usuario ficticio creado en G2-25 permanecía registrado con el correo qa.g225@example.com. Posteriormente, se ejecutó una solicitud POST /api/v1/users intentando crear otro usuario con el mismo correo electrónico. El backend respondió HTTP 409 (Conflict), con código de error CONFLICT y el mensaje "El correo ya está registrado". Se confirmó que la API detecta y rechaza la creación de usuarios con correos duplicados en la lista temporal evaluada. La prueba se realizó utilizando el encabezado usuario-actual sin JWT, vulnerabilidad de autenticación documentada previamente en G2-10.	PASS	Evidencias
31	**MEDIA**	G2-28	API / Validación	Provocar un error de validación	Devuelve 422	Se realizó una solicitud POST /api/v1/users intentando crear un usuario con datos inválidos, enviando el campo correo como un valor numérico (12345) en lugar de una cadena de texto. El backend respondió HTTP 422 (Unprocessable Content), con código VALIDATION_ERROR y detalle STRING_TYPE (Input should be a valid string). Se confirmó que la API rechaza correctamente el tipo de dato inválido y devuelve un error de validación estructurado, sin completar la creación del usuario. La prueba se realizó utilizando el encabezado usuario-actual sin JWT, vulnerabilidad de autenticación documentada previamente en G2-10.	PASS	Evidencias
32	**MEDIA**	G2-29	API / JSON	Revisar la estructura de las respuestas	Respeta el formato JSON acordado	Se realizó una solicitud POST /api/v1/auth/login enviando un cuerpo JSON mal formado mediante curl. El backend respondió HTTP 422 (Unprocessable Content), con código VALIDATION_ERROR y detalle JSON_INVALID (JSON decode error). Se confirmó que la API detecta y rechaza correctamente solicitudes con sintaxis JSON inválida, devolviendo un mensaje de error estructurado sin exponer información sensible.	PASS	Evidencia
33	**MEDIA**	G2-30	API / Errores	Revisar la estructura de las respuestas de error	Mantiene el formato común definido para errores	Se revisaron las respuestas de error obtenidas durante las pruebas de los endpoints, incluyendo HTTP 401 (UNAUTHORIZED), HTTP 404 (NOT_FOUND), HTTP 409 (CONFLICT) y HTTP 422 (VALIDATION_ERROR). Se confirmó que las respuestas evaluadas mantienen una estructura JSON consistente, utilizando los campos status, error y meta. El objeto error contiene code, message y details, mientras que meta incluye timestamp, path y request_id. Se verificó que el formato permite identificar claramente el tipo de error y su contexto. El escenario cumple con el comportamiento esperado para los errores evaluados; no se han comprobado todos los posibles errores del sistema.	PASS — Errores evaluados	Evidencias
34	**MEDIA**	G2-33	OpenAPI	Comparar endpoints documentados contra los endpoints reales	OpenAPI coincide con el comportamiento real del sistema	Se verificó la disponibilidad de la documentación mediante Swagger (/api/v1/docs) y OpenAPI (/api/v1/openapi.json), confirmando que ambos recursos son accesibles y que el documento OpenAPI responde HTTP 200. Se revisaron los endpoints publicados y sus respuestas documentadas. En POST /api/v1/users se identificaron los códigos 201 y 422, pero no se documenta HTTP 409 (Conflict), obtenido durante la prueba G2-27. Asimismo, la respuesta HTTP 201 presenta un ejemplo genérico de tipo string, aunque el endpoint devuelve un objeto JSON. También se observó que HTTP 401 en el endpoint de autenticación aparece como Undocumented. Por lo tanto, la documentación está disponible, pero presenta inconsistencias y omisiones respecto al comportamiento real de la API.	PARCIAL — Documentación incompleta	Evidencias

35	**BAJA**	G2-31	Paginación	Solicitar un listado con varias páginas	page, per_page, total y data funcionan correctamente	Se revisó el endpoint GET /api/v1/users mediante Swagger y el código fuente de users.py. Se confirmó que el único parámetro disponible es el encabezado usuario-actual y que no existen parámetros de paginación como page, limit u offset. Adicionalmente, se verificó que la función listar_usuarios() devuelve directamente la lista completa de usuarios, sin aplicar límites ni dividir los resultados en páginas. Por lo tanto, no fue posible validar el funcionamiento de la paginación. El escenario queda bloqueado debido a que esta funcionalidad no está implementada en el endpoint evaluado.	PENDIENTE	
36	**BAJA**	G2-32	Fechas	Revisar las fechas devueltas por la API	Utilizan ISO 8601 y la zona horaria acordada	Se verificó el formato de las fechas devueltas por la API. En la respuesta de GET /api/v1/users se identificó el campo creado_en con el valor 2026-06-22T00:00:00Z, correspondiente al formato ISO 8601 en UTC. Adicionalmente, se revisó el código encargado de construir las respuestas de error y se confirmó que el campo meta.timestamp se genera mediante datetime.now(timezone.utc).isoformat(). Se comprobó que los campos de fecha evaluados utilizan un formato compatible con ISO 8601. La verificación se limita a los campos identificados y no abarca todas las posibles fechas del sistema.	PASS — Campos evaluados	Evidencias
37	**BAJA**	G2-34	Login / Seguridad	Realizar 5 intentos fallidos consecutivos de login	El usuario queda bloqueado durante 15 minutos	En models.py se identificaron los campos intentos_fallidos y bloqueado_hasta. En registro.py se encontraron las funciones registrar_login_fallido(), contar_intentos_fallidos() y esta_bloqueado(), así como una prueba automatizada en test_errores.py que verifica el umbral de cinco intentos durante una ventana de 15 minutos. Sin embargo, en el endpoint POST /api/v1/auth/login no se identificaron llamadas a estas funciones ni actualizaciones del contador de intentos fallidos. El código revisado devuelve directamente HTTP 401 cuando la contraseña es incorrecta. Por lo tanto, aunque existen componentes auxiliares para implementar el bloqueo, no se encontró evidencia de su integración al flujo real de autenticación. El escenario no cumple según la revisión de código; queda pendiente una comprobación funcional de extremo a extremo.	NO CUMPLE — Integración de bloqueo no implementada en el login revisado.	Evidencias
38	**BAJA**	G2-36	Auditoría	Realizar una acción sensible	El acceso o acción queda registrado según corresponda	Se revisó la implementación del registro de auditoría. En registro.py se identificaron funciones para registrar acciones e intentos fallidos de autenticación, incluyendo registrar_accion() y registrar_login_fallido(). También se encontraron referencias a estas funciones en test_errores.py. Sin embargo, las búsquedas realizadas no identificaron llamadas a registrar_accion() ni a registrar_login_fallido() desde el endpoint de autenticación. Adicionalmente, se verificó que auth.py registra las sesiones exitosas mediante SesionToken, almacenando el hash del token, usuario, expiración, IP y user-agent, y actualizando el último acceso del usuario. Esto confirma que existe un registro de sesiones, pero no permite demostrar una auditoría completa de los eventos de autenticación. El escenario queda parcialmente evaluado por falta de evidencia de integración de las funciones de auditoría en el flujo real.	PARCIAL — Auditoría no integrada completamente	Evidencias

Prioridad ALTA: validar primero por impacto en seguridad y funcionamiento.
Prioridad MEDIA: validar contrato, comportamiento e integración de la API.
Prioridad BAJA: validaciones complementarias que igualmente deben revisarse antes del cierre.